

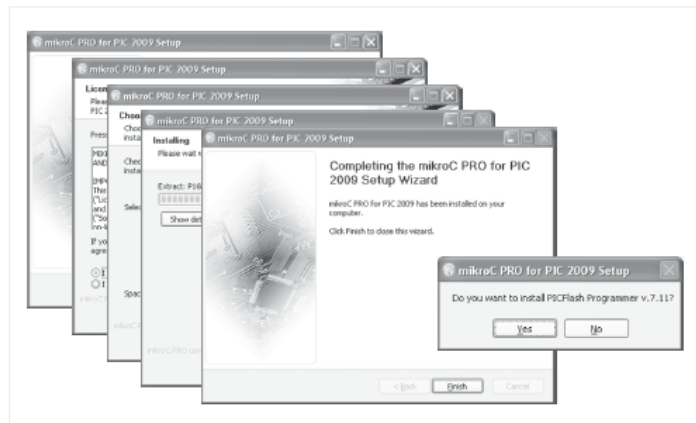
2.3 COMPILER MIKROC PRO FOR PIC

MIKROELEKTRONIKA

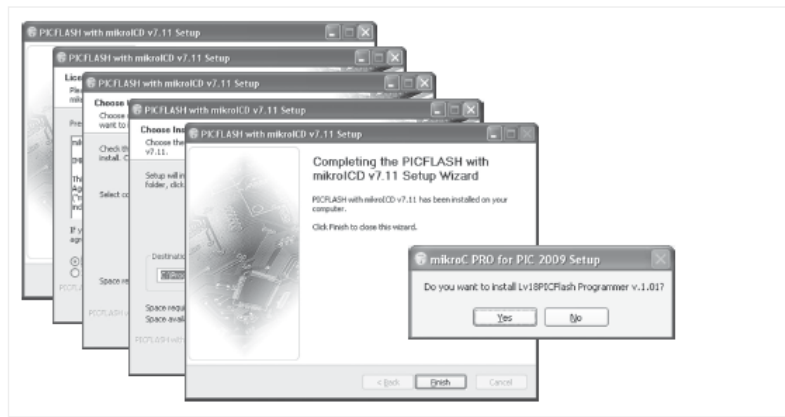
The first thing you need to write a program for the microcontroller is a PC program which understands the programming language you use, C in this case, and provides a window for writing program. Besides, the software must 'know' the architecture of the microcontroller in use. In this case, you need a compiler for C language.

There is no compiler to be used for only one concrete microcontroller as there is no compiler to be used for all microcontrollers. It's all about software used to program a group of similar microcontrollers of one manufacturer. This book gives description of the **mikroC PRO for PIC compiler**. As the name suggests, the compiler is intended for writing programs for PIC microcontrollers in C language. It is provided with all data on internal architecture of these microcontrollers, operation of particular circuits, instruction set, names of registers, their accurate addresses, pinouts etc. When you start up the compiler, the next thing to do is to select a chip from the list and operating frequency and of course – to write a program in C language.

The installation of *mikroC PRO for PIC* is similar to the installation of any Windows program:

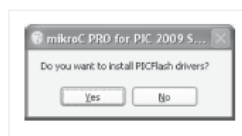


Wizard is in charge of the whole procedure, you should just click options *Next*, *OK*, *Next*, *Next*... All in all, the same old procedure except for the last option 'Do you want to install PICFLASH v7.11 programmer?'. Why is that? The compiler's task is to convert a program written in C language into Hex code. What comes next is to program the microcontroller. It's the responsibility of hardware and software, not any software, but *PICFLASH v7.11 programmer*. Install it! Of course: *Next*, *OK*, *Next*, *Next*...

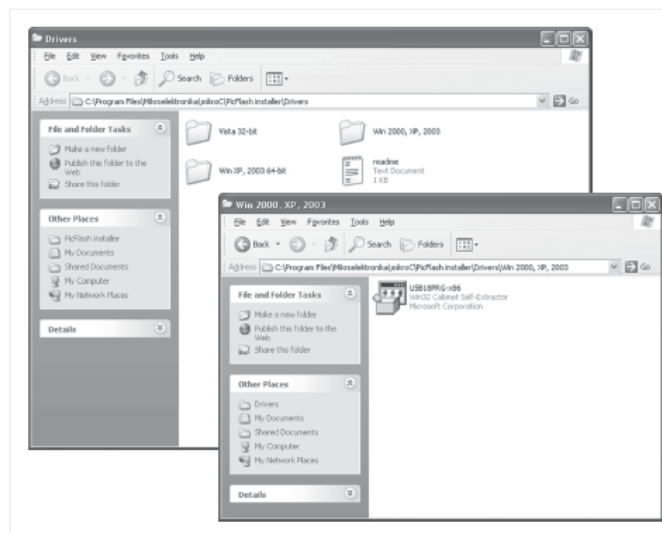


When the installation of this software is complete, you will be prompted for the installation of another similar program. It is software for programming a special group of PIC microcontrollers which operate in low consumption mode (3.3 V). Skip it...

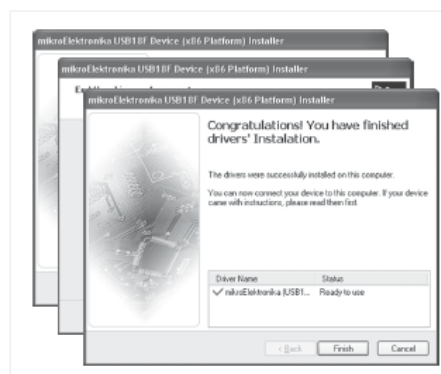
The last step – driver installation!



Driver is a program which enables the programmer's software you have just installed on your PC and hardware to communicate with each other. If you have followed instructions so far you will definitely need it. Click Yes.



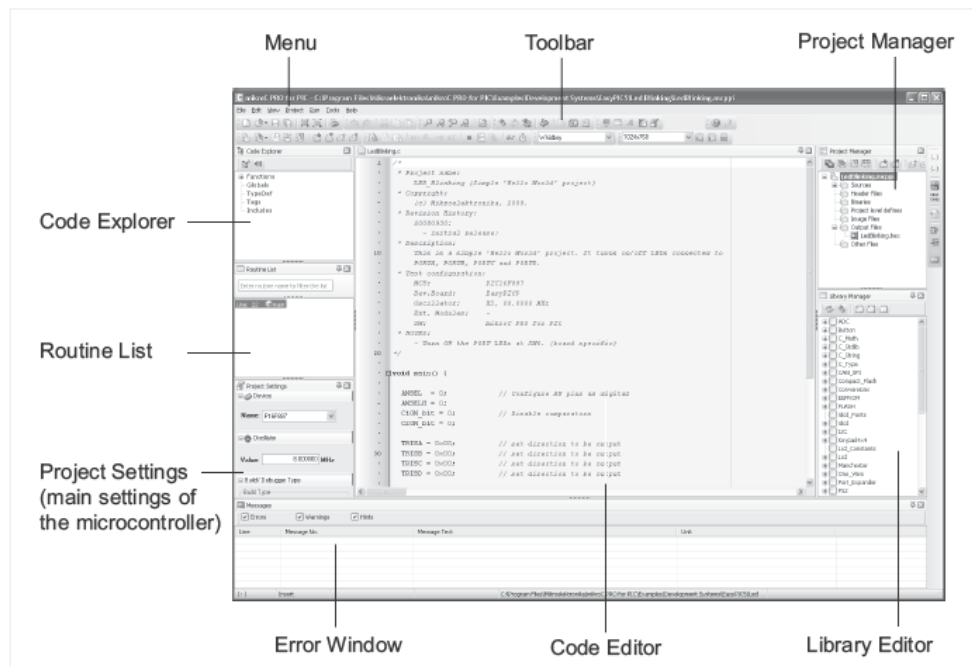
Type of drivers depends on operating system in use. Select the appropriate folder and start up installation.



Now you are safe, just keep on clicking *Next*, *OK*, *Next*, *Next*...

IDE FOR MIKROC PRO FOR PIC

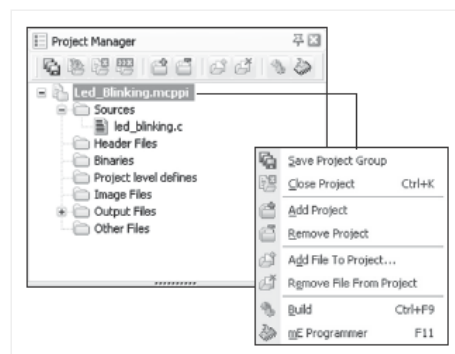
This is what you get when you start up *IDE for mikroC PRO for PIC* for the very first time:



Unfortunately, a detailed description of all the options available in this compiler would take too much of our time, so that we are going to skip it. Instead, we are going to describe only the process of writing a program in C language, simulator checking as well as its loading into the microcontroller memory. For more information refer to help [F1].

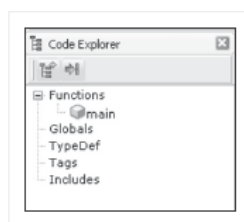
PROJECT MANAGER

A program written in mikroC compiler is not a separate document, but part of a project which includes Hex code, assembly code, header and other files. Some of them are created during the operation of compiler, while some are imported from other programs. However, the Project Manager Window enables you to handle them all. It is sufficient to right click any folder and select the option you need for your project.



CODE EXPLORER

The *Code Explorer* window enables you to easily locate functions and procedures within long programs. For example, if you look for a function used in the program, just double click its name in this window, and the cursor will be automatically positioned at appropriate point in the program.



PROJECT SETTINGS

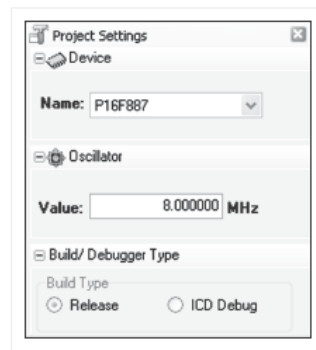
Device – When you select the microcontroller, the compiler automatically knows which definition file, containing all SFR registers for specific MCU, their memory addresses and similar, to use.

Oscillator – This option is used to select the operating speed of the microcontroller. On the basis of it, the compiler makes changes in the configuration word. The operating speed is set so as to enable the microcontroller's internal oscillator to operate with selected quartz crystal.

Build type – release After the process of compilation is complete, the compiler has no influence on the program execution. For the purpose of debugging, a software simulator can be used.

In order to enable the compiler to operate successfully, it is necessary to provide it with basic information on the microcontroller in use as well as with the information on what is expected from it after the process of compilation:

Build type – ICD debug: When the process of compilation is complete and the microcontroller is programmed, the compiler remains connected to the microcontroller and still can affect its operation. The connection is established via programmer which is connected to the PC via USB cable. A software making all this work is called the ICD (*In Circuit Debugger*). It enables the program to be executed step by step and provides an access to the current content of all registers of the microcontroller. Simulation is not carried out, their contents is literally read in true MCU controlling true device.

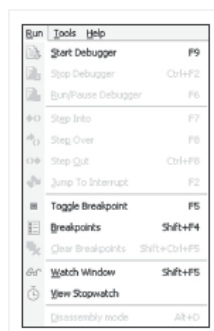


CODE EDITOR

A Code Editor is a central part of the compiler window used for writing a program. A large number of options used for setting its function and layout can be found in the *Tools/Options* menu [F12].

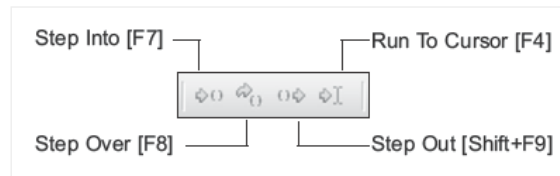
SOFTWARE SIMULATOR

Prior to starting up the simulator, select the appropriate mode in the *Project Settings* Window (*Build type – release*) and click the **Run /Start Debugger** option.



The compiler will be automatically set in simulation mode. As such, it monitors the state of all register bits. It also enables you to execute the program step by step while monitoring the operation of the microcontroller on the screen (i.e. simulation of operation).

A few icons, used only for the operation of this simulator, will be added to the toolbar when setting the compiler in this mode.



They have the following meanings:

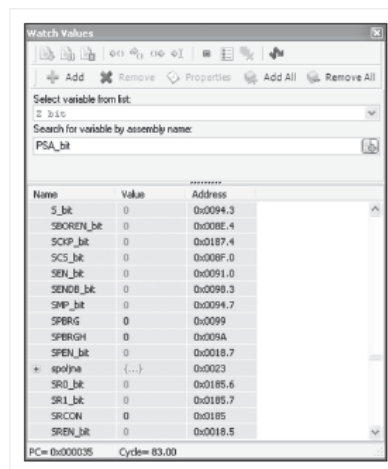
Step Into – Click on this icon executes one program line in which the cursor is positioned.

Step Over – This command is similar to the previous one. If the cursor is positioned in the line which calls a program routine than it will be executed first and the program proceeds with execution at the first next program line. It seems as if one program line is skipped even though the whole routine is executed. As a result, the state of registers change. This command is commonly used when it is necessary to speed up the execution of long program loops.

Run To Cursor – This command is used to execute a particular part of the program, i.e. from the last executed line -to the line in which the cursor is placed.

Step out – By clicking this icon, the program exits routine being currently executed.

The simulator and debugger have the same function to monitor the state of registers during program execution. The difference is that the simulator executes the program on the PC, while the debugger uses a true microcontroller. Any change of a pin logic state is reflected on appropriate register (port). As the *Watch Window* allows you to monitor the state of all registers it is easy to check whether a pin is set to zero or one. In order to activate this window it is necessary to select **View/Windows** and click the **Watch Values** option. Then you can make a list of registers the state of which you want to monitor.



If you want to find out how long it takes for the microcontroller to execute a part of the program, select the **Run/View Stopwatch** option. A window as shown in figure on the right will appear. Do you know how the stopwatch works? Well, it's as simple as that.

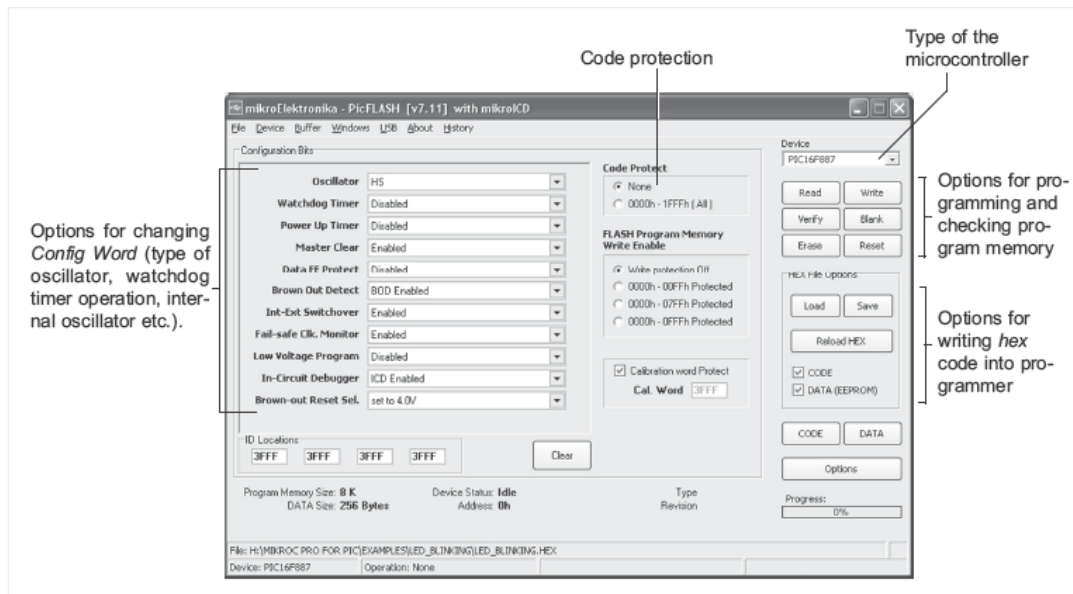


COMPILER'S TOOLS

This compiler provides special tools which considerably simplify the process of writing a program. All these tools are available from the *Tools* menu. In the following text we are going to give a brief description of all of them.

PICFLASH PROGRAMMER

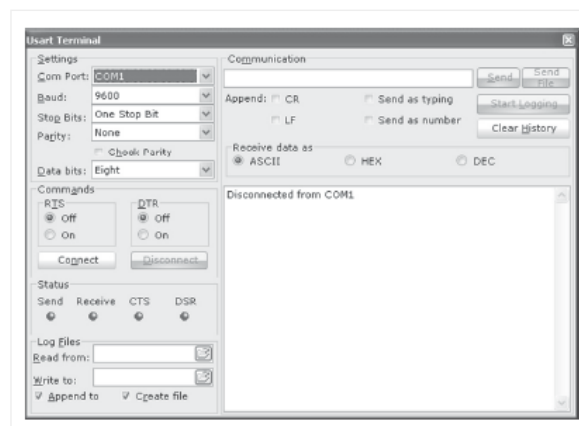
PICflash programmer is a stand-alone program which can operate independently of the compiler, i.e. it can be used as a separate program. However, in this case, its operation is closely related to the operation of the compiler so that it can be activated from within the compiler itself. If installed, the PIC flash programmer is activated by selecting **Tools/me_Programmer** or pressing [F11]. A window that appears contains options to be used for the process of programming microcontrollers.



It's the right time to explain the operation of the programmer. As you know, the compiler is a software which compile the program written in a higher programming language into executable code, i.e. Hex code. That's the code the microcontroller understands and executes. The programmer, which loads this code into the chip, is comprised of software and hardware together called – PICflash programmer. Programmer's hardware provides all necessary voltage levels and socket for placing the microcontroller in. Programmer's software is installed on the PC and is used to pass on the Hex code to hardware over USB cable. This book provides discussion on the software only.

USART TERMINAL

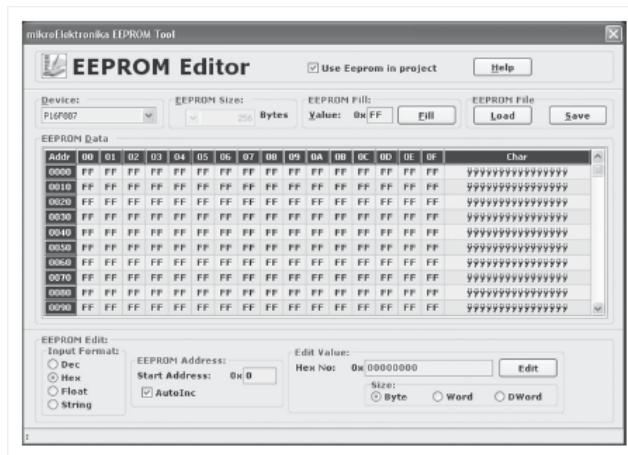
The USART terminal is a replacement for the standard *Windows Hyper Terminal*. It can be used for checking the operation of the microcontroller which uses USART communication. Such a microcontroller is built in a device and connected to the RS232 connector on PC over serial cable. The USART terminal window, shown on the right, contains options for setting serial communication and for displaying sent/received data.



EEPROM EDITOR

If you select the **EEPROM Editor** option from the **Tools** menu, a window, as shown in figure on the right, will

appear. This is how the EEPROM memory within the microcontroller looks like. If you want to change its contents after loading the program into the microcontroller this is the right place to do it. If a new content is a data of specific type (*char*, *int* or *double*), then you should select it, enter the value in the *Edit Value* field and click *Edit*. Then click the *Save* button to save the data as a document with .hex extension. If the *Use EEPROM in Project* option is active, the data will be automatically loaded into the chip during the process of programming.



ASCII CHART

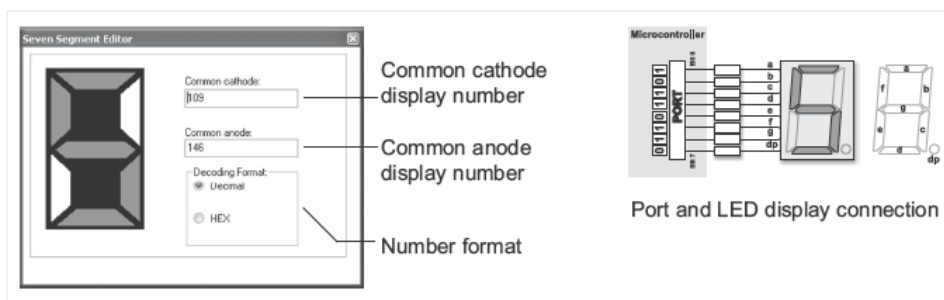
If you need numerical representation of any ASCII character, just select the appropriate option from the *Tools* menu and the table, as shown in figure below, will appear.

As seen, the characters representing numbers have curious equivalents. For this reason, program command for displaying number 7 on an LCD display will not display anything similar this number. Instead, the equivalent of the command BEL will be displayed. If you send the same number as a character, you will get the expected result – the number 7. Accordingly, if you want to display a number without previously converting it into character, then it is necessary to add the number 48 to each digit the number consists of.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2	SPC	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL
8	€	□	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊
9	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊
A	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊
B	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊
C	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊
D	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊
E	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊
F	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊	◊

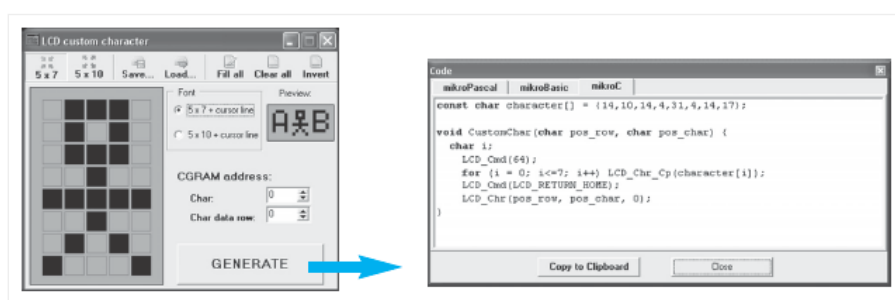
SEVEN SEGMENT EDITOR

A seven segment editor enables you to easily find out which number is necessary to be set on an output port in order to display a desired symbol. Of course, what goes without saying is that port pins must be connected to display segments properly. You just have to place the cursor on any display segment and click it. The number that you should copy to the program will be shown immediately. That's all.



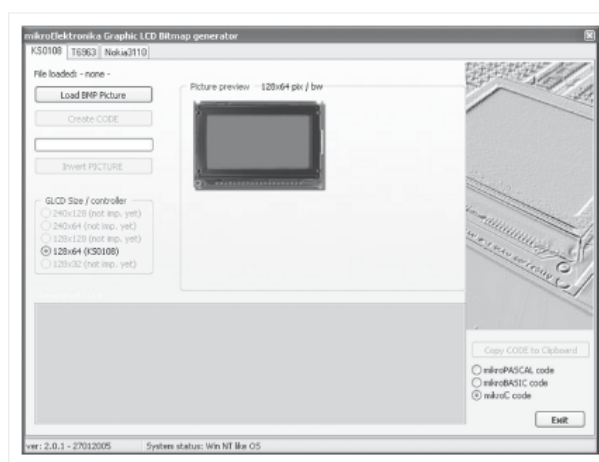
LCD CUSTOM CHARACTER

Apart from the standard characters, the microcontroller can also send characters created on your own to a display. By selecting the *LCD custom character* tool you will spare yourself from tedious work on creating functions for sending appropriate code to a display. Just create a symbol by clicking small squares in the *LCD custom character* window, select position and row and click the *GENERATE* button. The required code appears in another window. No more clicks are needed. *Copy to Clipboard – Paste...*



GRAPHIC LCD BITMAP GENERATOR

This is another irreplaceable tool in the event that the microcontroller you are writing program for uses graphic LCD display (*GLCD*). This tool enables you to display any bitmap easily. In order to take advantage of it, select **Tools/Glcd Bitmap Editor** and appropriate window appears. Select type of display to be used and load a bitmap. The bitmap must be monochromatic and in resolution specified (128 x 64 pixels in this example). Further procedure is the same as in the example above *Copy to Clipboard...*



A code generated using tools for controlling LCD and GLCD displays contains functions of the Lcd library. If you use them in the program, don't forget to check the box next to this library in the Library Manager window so as to enable the compiler to recognize its functions correctly.

LIBRARIES

One of the most useful options of this program is *Library Manager* and surely deserves our attention.

It is previously mentioned that the main advantage of the higher programming languages such as C is that these enable you to use the knowledge and work of other people. Function libraries are the best example of it. If you need a function to perform certain task while writing a program, you just have to look for it within some of the libraries which are integrated in the compiler and use it. For example, if you need a function to generate sound on some of the pins, open the *Sound* library in the *Library Manager* window and double click the appropriate function *Sound_Play*. A detailed description of this function appears on the screen. Copy it to your program and set appropriate parameters. If this library is checked, its functions will be automatically recognized during the process of compiling so that it is not necessary to use the **#include** command.



STANDARD ANSI C LIBRARIES

Standard ANSI C libraries includes standard functions of C language:

LIBRARY	DESCRIPTION
ANSI C Ctype Library	Mainly used for testing or data conversion
ANSI C Math Library	Used for floating point mathematical operations
ANSI C Stdlib Library	Contains standard library functions
ANSI C String Library	Used to perform string and memory manipulation operations

MISCELLANEOUS LIBRARIES

Miscellaneous libraries contain some of the general-purpose functions which are not included in standard ANSI C libraries:

LIBRARY	DESCRIPTION
Button Library	Used for a project development
Conversion Library	Used for data type conversion
Sprintf Library	Used for easy data formatting
PrintOut Library	Used for easy data formatting and printing
Time Library	Used for time calculations (UNIX time format)
Trigonometry Library	Used for fundamental trigonometry functions implementation
Setjmp Library	Used for program jumping

HARDWARE SPECIFIC LIBRARIES

Hardware specific libraries include functions intended to be used for controlling the operation of various hardware modules:

LIBRARY	DESCRIPTION
ADC Library	Used for A/D converter operation
CAN Library	Used for operation with CAN module
CANSPI Library	Used for operation with external CAN module (MCP2515 or MCP2510)
Compact Flash Library	Used for operation with <i>Compact Flash memory cards</i>
EEPROM Library	Used for operation with built-in EEPROM memory
EthernetPIC18FxxJ60 Library	Used for operation with built-in Ethernet module
Flash Memory Library	Used for operation with built-in Flash memory
Graphic Lcd Library	Used for operation with graphic LCD module with 128×64 resolution
I2C Library	Used for operation with built-in serial communication module I2C
Keypad Library	Used for operation with keyboard (4×4 push buttons)
Lcd Library	Used for operation with LCD display (2×16 characters)
Manchester Code Library	Used for communication using <i>Manchester code</i>
Multi Media Card Library	Used for operation with multimedia MMC flash cards
One Wire Library	Used for operation with circuits using <i>One Wire</i> serial communication
Port Expander Library	Used for operation with port expander MCP23S17
PS/2 Library	Used for operation with standard keyboard PS/2
PWM Library	Used for operation with built-in PWM module
RS-485 Library	Used for operation with modules using RS485 serial communication
Software I2C Library	Used for I2C software simulation
Software SPI Library	Used for SPI software simulation
Software UART Library	Used for UART software simulation
Sound Library	Used for audio signal generation
SPI Library	Used for operation with built-in SPI module
SPI Ethernet Library	Used for SPI communication with ETHERNET module (ENC28J60)
SPI Graphic Lcd Library	Used for 4-bit SPI communication with graphic LCD display
SPI Lcd Library	Used for 4-bit SPI communication with LCD display (2×16 characters)
SPI Lcd8 Library	Used for 8-bit SPI communication with LCD display
SPI 6963C Graphic Lcd Library	Used for SPI communication with graphic LCD display
UART Library	Used for operation with built-in UART module
USB Hid Library	Used for operation with built-in USB module

ACCESSING INDIVIDUAL BITS

The *mikroC PRO for PIC* compiler allows you to access individual bits of 8-bit variables by their name or position in the byte:

```
1 INTCON.B0 = 0; // Clear bit 0 of the INTCON register
2 ADCON0.F5 = 1; // Set bit 5 of the ADCON0 register
3 INTCON.GIE = 0; // Clear Global Interrupt Bit (GIE)
```

SBIT TYPE

The *mikroC PRO for PIC* compiler has an sbit data type which provides access to registers, SFRs, variables, etc. In order to declare a bit of a variable, it is sufficient to write:

```
1 extern sbit Some_Bit; // Some_Bit is defined
2 char MyVar;
3 sbit Some_Bit <b>at</b> MyVar.F0; // This is where Some_Bit is declared
4 ...
5 void main() {
6 ...
7 }
```

If you declare an **sbit** variable in a unit so as to point it to a specific bit of SFR register, it is necessary to use the

keyword **sfr** in declaration, because you are pointing it to the variable defined as sfr variable:

```
1 extern sfr sbit Abit; // Abit is precisely defined
2 ...
3 sbit Abit at PORTB.F0; // Now, Abit is declared
4 void main() {
5 ...
6 }
```

BIT TYPE

The *mikroC PRO for PIC* compiler provides a bit data type that may be used for variable declarations. It cannot be used for argument lists and function-return values.

```
1 bit bf; // Valid bit variable
2 bit *ptr; // Invalid bit variable. There are no pointers to bit variables
```



2.3 Compiler mikroC pro for PIC by MikroElektronika is licensed under a Creative Commons Attribution 4.0 International License, except where otherwise noted.